HPCA 2026

DSASSASSIN

Cross-VM side-channel attacks by exploiting Intel’s Data Streaming Accelerator

Ben Chen · Kunlin Li · Shuwen Deng · Dongsheng Wang · Yun Chen

HKUST (Guangzhou) · Tsinghua University

Presented by Siddharth Banakar and Dinesh Samala · 29 October 2026

The wall has a crack

Address isolation still leaves the shared activity visible



The leakage is about resource use — not access to victim memory.

Meet the machine

Memory offload creates a new place to observe

Common operations

memcpy · memset

compare · checksums

→

DSA offload

descriptor submission

asynchronous execution

→

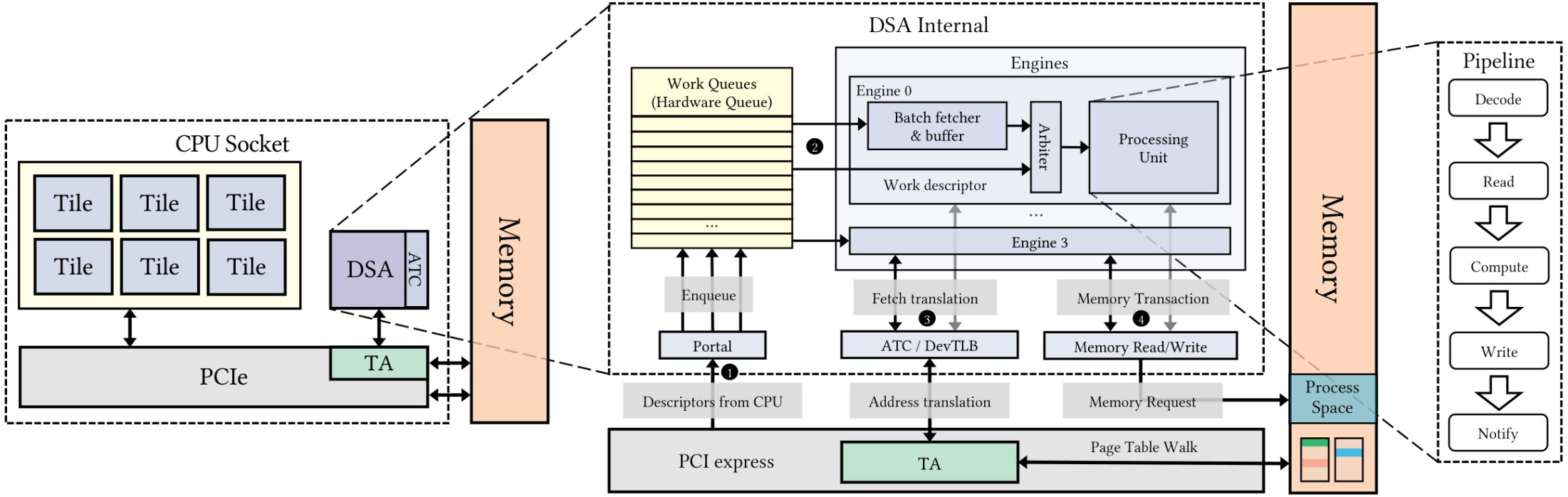
Observable state

translation cache

queue occupancy

Moving work off the CPU moves the side-channel surface with it.

A DSA operation is a descriptor-driven pipeline



ORIGINAL FIGURE 1 · CPU → portal → work queue → engine → completion

Follow one descriptor through the engine and its memory-translation path.

The device keeps its own translation cache

Device request

virtual address + PASID

→

DevTLB / ATC

hit → reuse translation

→

Host memory

physical transaction

Miss → Translation Agent / IOTLB → page-table walk if needed

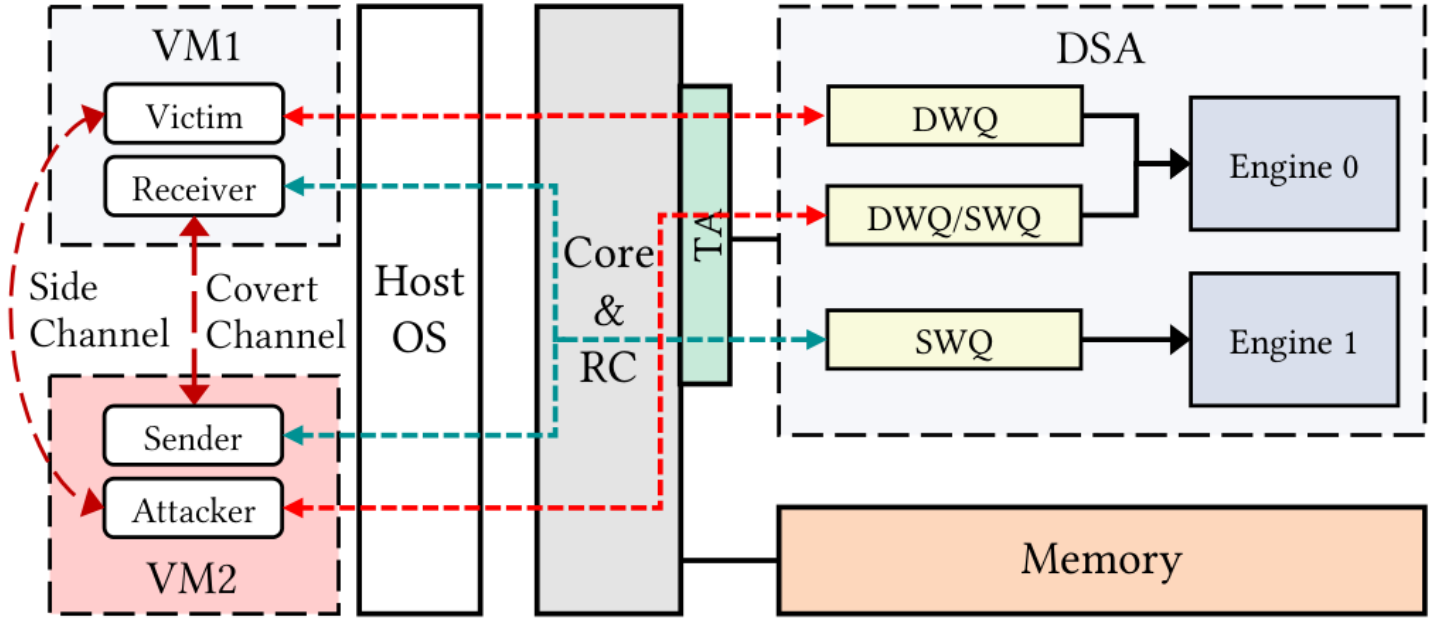
Shared replacement capacity can leak activity without exposing page contents.

The attacker needs DSA access and co-location

Assumed	Not needed
DSA-enabled Xeon + IDXD + VT-d scalable mode	A shared CPU core
Victim actually uses DSA	Shared victim memory
Attacker may submit descriptors (CAP_SYS_RAWIO)	Permission to read victim addresses
Shares the engine (DevTLB) or the SWQ	Root / authority to reconfigure host DSA
May set its own engine / queue affinity	

“Unprivileged” still depends on pre-provisioned device access and co-location.

Two channels cross two different boundaries



Two sharing requirements

Red: DevTLB timing
Same engine; queues may differ.

Teal: SWQ contention
Same shared work queue.

The shared resource determines which boundary the attacker can observe.

Characterization counters ≠ attack signals

Reverse engineering · privileged PMU

EV_ATC_ALLOC

EV_ATC_NO_ALLOC

EV_ATC_HIT_PREV

Attack signal · unprivileged

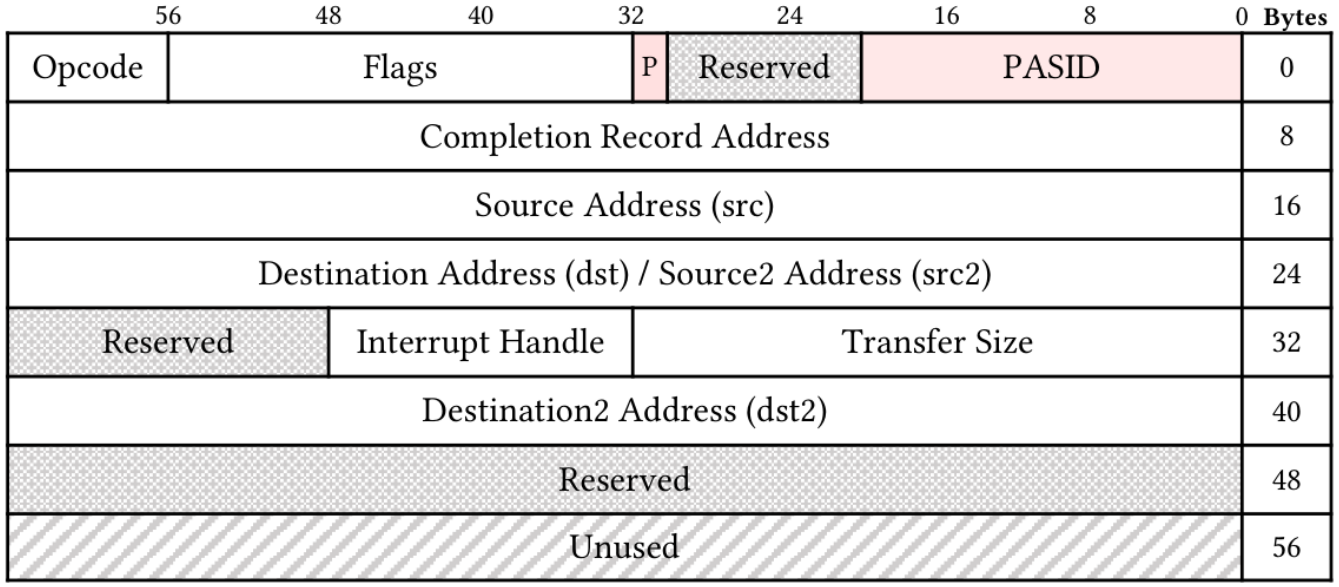
DevTLB · own completion time (rdtsc)

SWQ · ENQCMD result via EFLAGS.ZF

Platforms: Xeon Platinum 8468V (local) · 8475B (Alibaba cloud) · Linux 6.11 · IDXD · VT-d scalable mode · SR-IOV (DevTLB setup) / Scalable IOV (shared SWQ)

PMU access supports the explanation; it is not needed for the attack loop.

Clue #1 — the opcode chooses which slot you touch



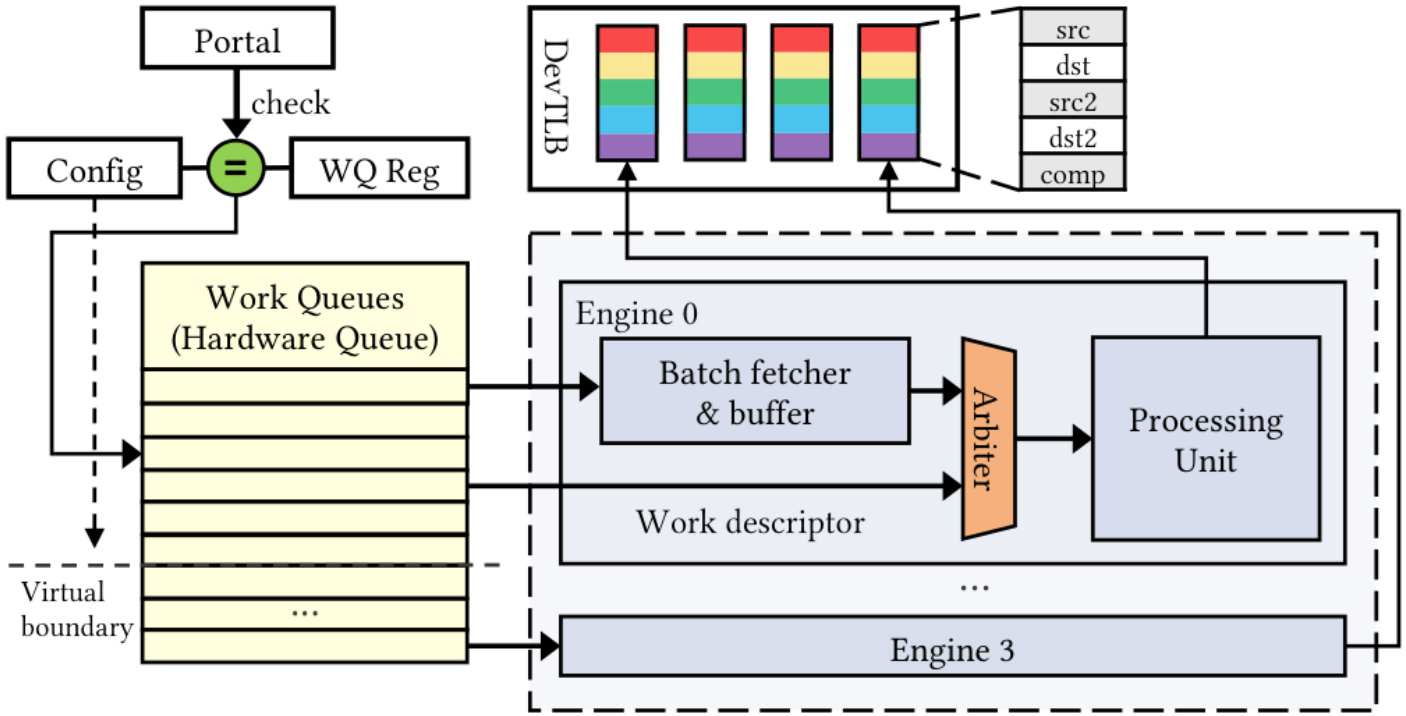
SEMANTIC FIELD → SLOT

OPERATION	FIELDS TOUCHED
NOOP	comp
Compare	src, src2, comp
Copy	src, dst, comp
Dualcast	src, dst, dst2, comp

Completion writing is enabled in these examples.

The same bytes at offset 24 mean src2 for compare and dst for copy.

The reveal — five drawers, one translation each



Read the original structure

One engine owns five slots:
src · src2 · dst · dst2 · comp

One translation per slot.

A new page in the same field replaces the previous entry.

The slots cache translations; shared replacement capacity exposes activity.

Proof of “one slot” — base → offset → base

LISTING 2 · PAPER EXCERPT

```
probe_noop(base); // base is 4KB aligned
probe_noop(base + OFFSET); // try evicting
probe_noop(base); // probe if base is present
```

OFFSET FROM BASE	FINAL ACCESS TO BASE	WHAT IT ESTABLISHES
Within the same 4 KiB page	Hit	Page-granular reuse
Another page	**Miss**	**One conflicting page replaces base**

For NOOP, base is the attacker’s completion-record page.

The drawers are independent — even sharing bits

LISTING 3 · PAPER EXCERPT

```
// different page
assert(PAGE_OF(src0) != PAGE_OF(dst0));
probe_memcpy(src0, dst0); // prime devtlb
// src1 is on a different page with the src
probe_memcpy(src1, dst0);
```

Change src; keep dst.

The destination translation survives.

LISTING 4 · PAPER EXCERPT

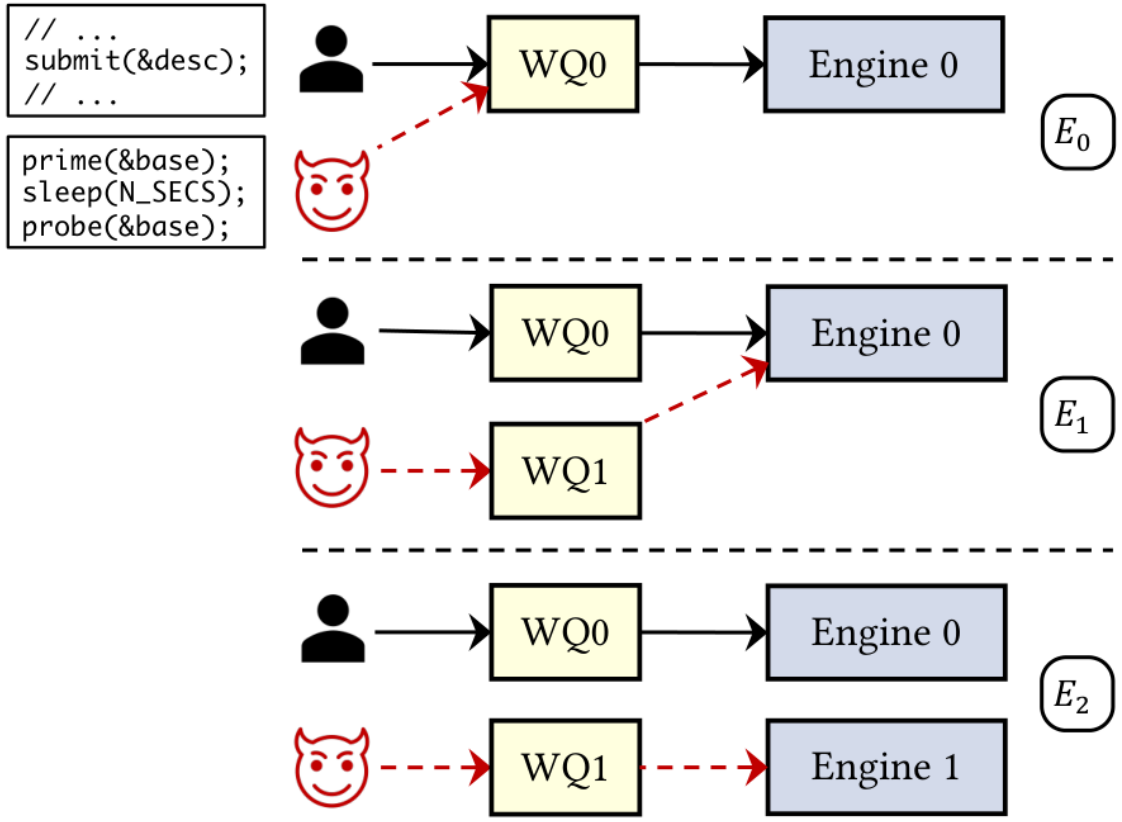
```
// src2 overlaps dst
probe_memcmp(src, src2);
// dst and src2 in same page
probe_memcpy(src, dst);
```

Same page / encoding position; different role.

Only src contributes the reuse hit.

Different semantic fields use different DevTLB slots.

The money shot — eviction follows the engine, not the queue



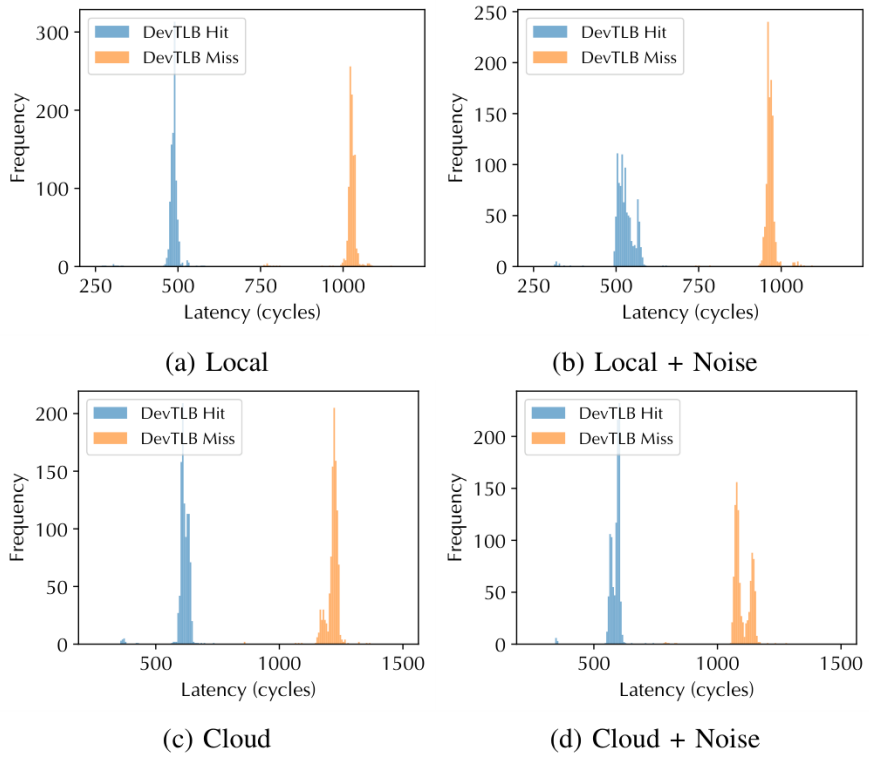
CASE	SHARED RESOURCE	PROBE
E0	same WQ + engine	Miss
E1	different WQs; same engine	Miss
E2	different engines	Hit

E1: separate queues still share DevTLB capacity.

E2: separate engines preserve the primed entry.

In these experiments, eviction followed the shared engine.

The clock still works — even under noise



Measured signal

Hit	≈ 500 cycles
Miss	≈ > 1000 cycles
Threshold	600 – 900

Noise shifts latency; the separation survives in every tested environment.

An unprivileged clock cleanly tells a hit from a miss.

Weapon #1 — Prime → Idle → Probe

PRIME: own completion page → IDLE: allow victim activity → PROBE: time own request

LISTING 1 · LINES 7–12 · PAPER EXCERPT

```
enqcmd(wq_portal, &desc);
// time polling for completion
start = rdtsc();
while (comp.status == 0);
end = rdtsc();
return end - start;
```

What the timer measures

ENQCMD returns first.

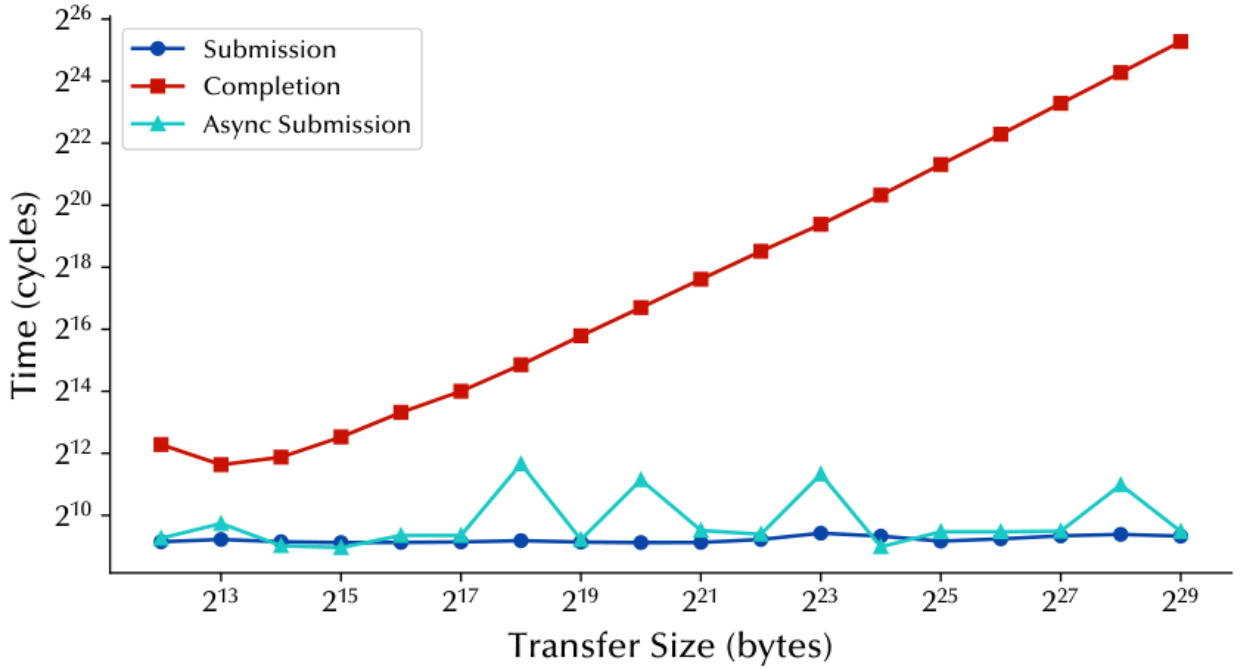
RDTSC brackets completion polling.

The result is the attacker's own completion duration.

Listing 1 times a compare helper; §V-B uses NOOP for the attack. The timing pattern carries over.

Slow completion suggests replacement; it does not reveal the victim's address.

The dead end that wasn’t — a flag talks



Two observables

Submission flat ≈ 700 cycles

Completion grows with size

Queue-full a visible flag

The SWQ channel uses ENQCMD / EFLAGS.ZF — no timing threshold at all.

Learning to hold the queue hostage

LISTING 5 · PAPER EXCERPT

```
enqcmd(wq_portal, &batch_desc);// (1)
_mm_sfence();
enqcmd(wq_portal, &desc); start = rdtsc();// (2)
while (desc_comp.status == 0);
latency = rdtsc() - start;
```

Repeat the experiment after swapping the two ENQCMD submission positions.

What the experiment found

Work completion latency barely changed when order was swapped.

Measured preference:
ordinary work over batch work.

A batch is an unreliable anchor.

Arbiter priority motivates a long ordinary copy for congestion.

The trap — leave exactly one seat open

LISTING 6 · PAPER EXCERPT

```
struct dsa_hw_desc desc_buf[] = {...};

void congest(int wq_size) {
    enqcmd(wq_portal, &mass_desc);
    for(int i = 1; i < wq_size - 1; i++)
        enqcmd(wq_portal, &desc_buf[i]);
}

int probe() { return enqcmd(wq_portal, &desc); }
```

Connect code to queue state

mass_desc = long ordinary copy.

wq_size − 2 additional descriptors.

Target: N − 1 occupied positions.

probe() reads acceptance / retry, not the operation's completion.

Paper observation: congestion became visible for copies ≥ 32 MiB. Descriptor preparation is omitted.

Keep the long copy active while leaving one admission position available.

Weapon #2 — Congest → Idle → Probe

1

Congest

Long memcpy + fillers → near-full SWQ

→

2

Idle

Hold the window; the victim may enqueue

→

3

Probe

Submit once, read EFLAGS.ZF

ZF = 0 → accepted (capacity available)

ZF = 1 → retry (full in this experiment)

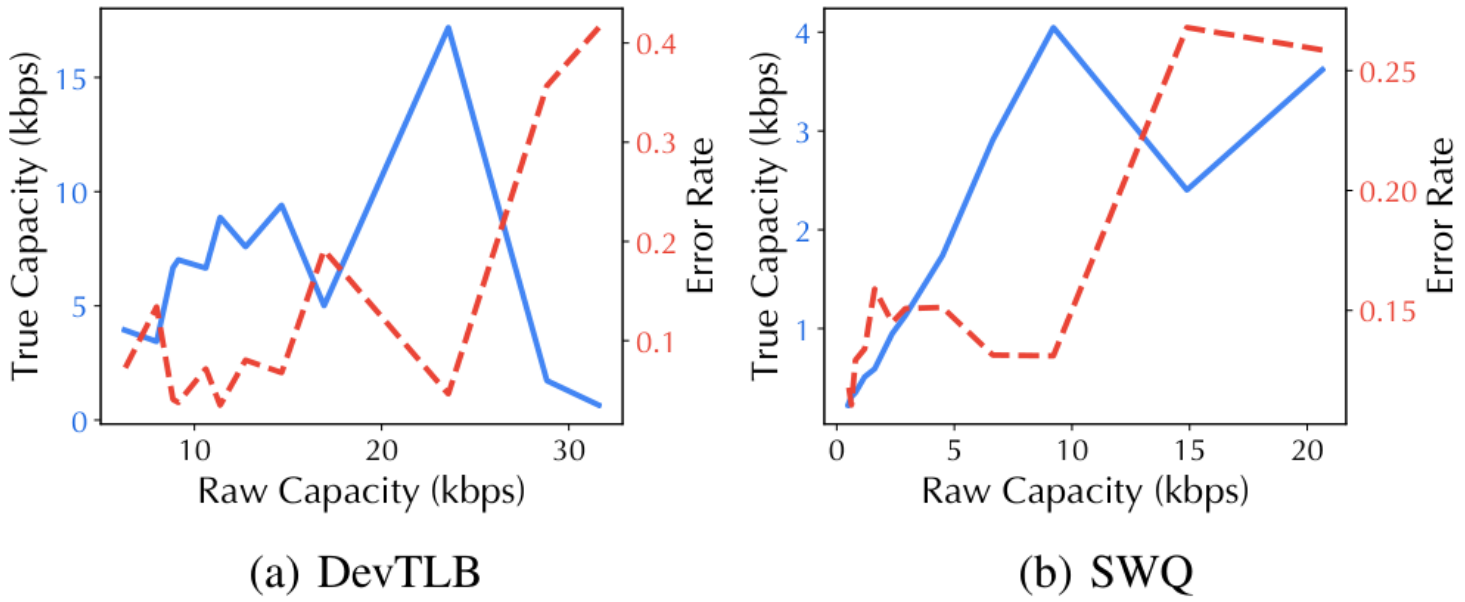
"Timer-free" describes the signal; the window still needs control.

Can it even carry a message?

PHASE	RECEIVER	SENDER
Synchronize	detect preamble	transmit consecutive 1 bits
Prepare	prime DevTLB / congest SWQ	wait for data slot
Encode	open observation window	1 → submit · 0 → idle
Decode	time completion / read ZF	advance to next slot

Covert-channel feasibility is a separate claim from inference accuracy.

17.19 kbps — and a bandwidth / error tradeoff



17.19 kbps

DevTLB

4.63% bit error

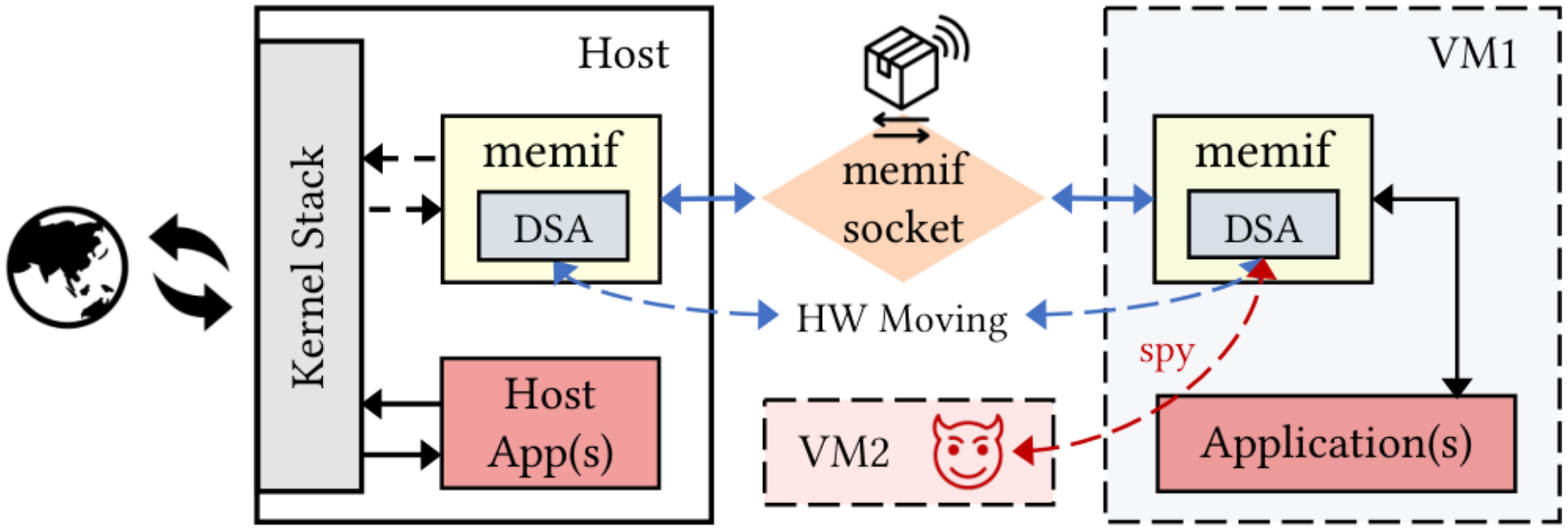
4.02 kbps

SWQ

13.11% bit error

Faster raw signalling loses useful throughput as the error rate climbs.

Attack #1 — websites leak through the stack



Website traffic → DSA-accelerated memif transfers → shared-engine activity → miss-count trace

The spy observes its own probes; the classifier supplies the website label.

The pipeline, laid bare — and it is closed-world

Sampling

probe every 10 µs

400 probes / slot

≈ 4 ms per slot

Model

2 × BiLSTM

attention + dropout

softmax classes

Dataset

100 sites × 200 traces

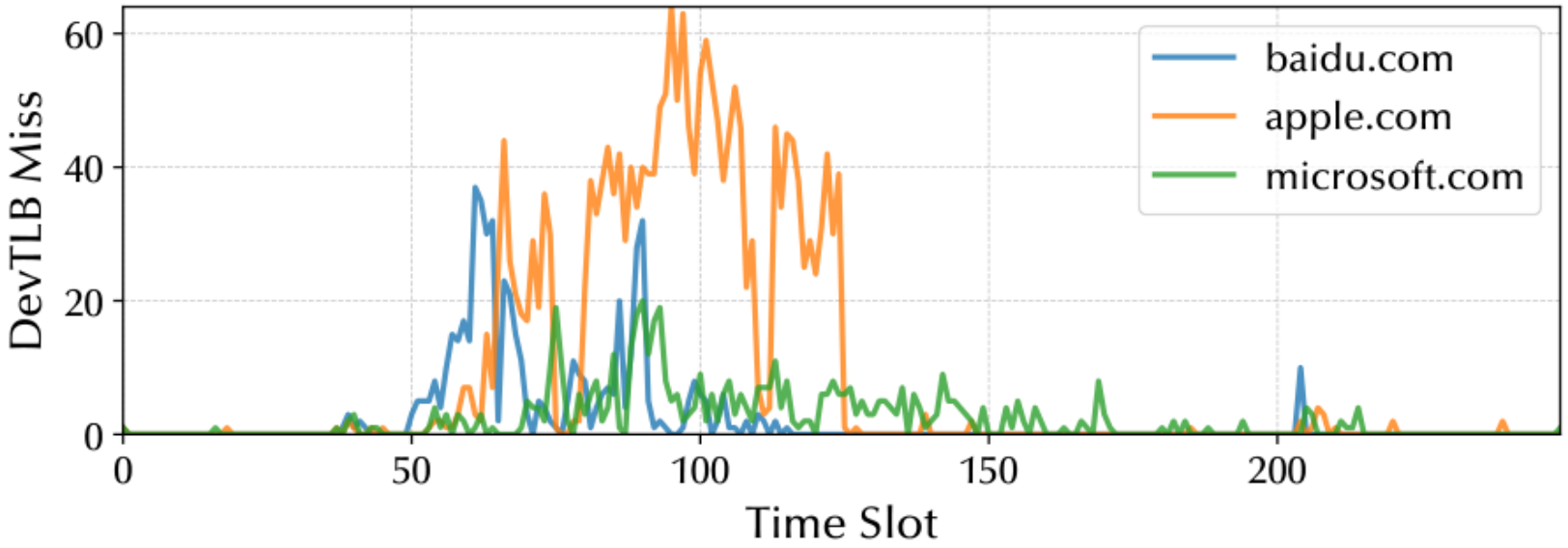
80 / 20 split

≈ 16,000 / 4,000

One feature per slot = the number of classified DevTLB misses.

The evaluated problem is closed-world classification over known labels.

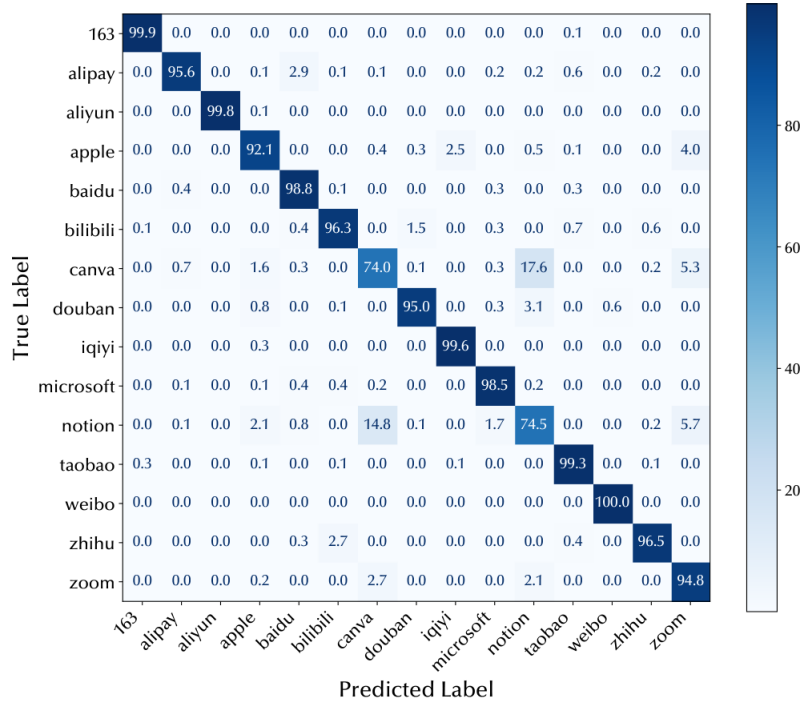
The traces really are different



Temporal shape is the feature; miss count is a proxy for DSA activity.

What it steals

85.73% — the number you must not misquote



85.73%

Full dataset

100 websites · 1% baseline

96.5%

Displayed subset

15 selected websites only

The headline is 85.73% on 100 sites; 96.5% is the 15-site subset.

What it steals

Attack #2 — keystrokes, via transparent offload

User input

interactive SSH event

→

Packet handling

OpenSSH mem* calls

→

DTO offload

eligible calls → DSA

→

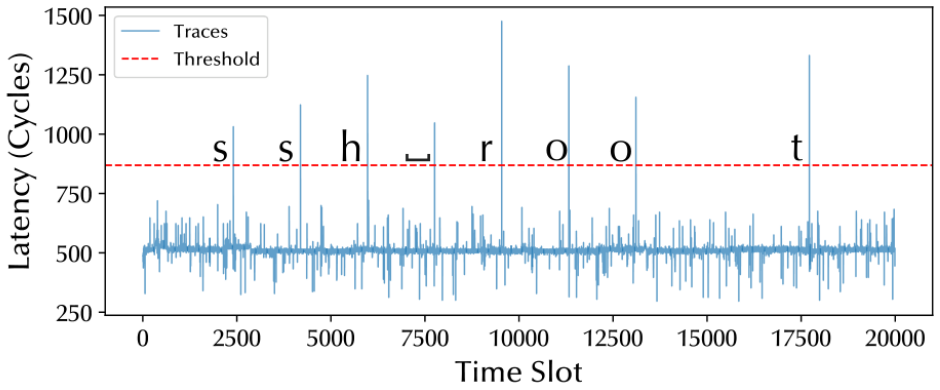
DSA observation

eviction or congestion

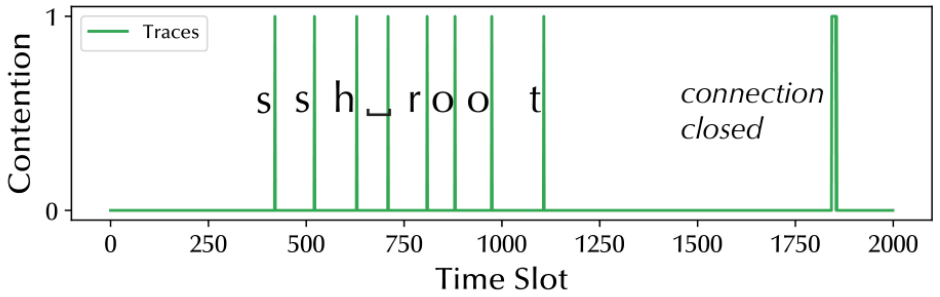
The demonstrated output is keystroke timing — not recovered command text.

What it steals

You can see the typing



(a) DevTLB traces during SSH input events.



(b) SWQ traces during SSH input events.

Read the panels

Top: completion latency

Bottom: queue contention

Labels: ground-truth input

Filtering affects event metrics; thresholds matter.

Both primitives reveal candidate keystroke events.

SWQ wins timing — and a caveat I owe you

PRIMITIVE	REPORTED F1	TIMESTAMP DEVIATION (Σ)
DevTLB	92.0%	5.29 ms
SWQ	**98.4%**	**1.21 ms**

Reporting caveat: the detection / FP / FN counts in § VI-C do not fully reconcile with the printed F1 (see backup).

Use the headline values with attribution; disclose the arithmetic gap.

What it steals

Attack #3 — fingerprinting the LLM next door

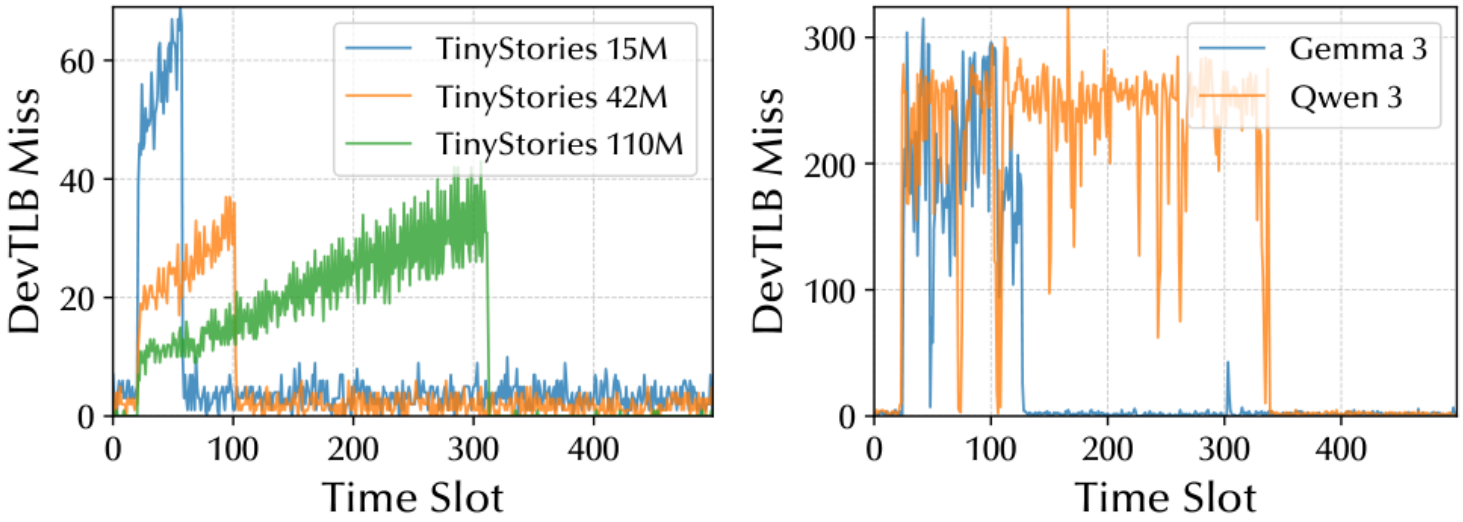
FAMILY	PARAMETER COUNTS	CHOICES
TinyStories	15M / 42M / 110M	3
LLaMA 2	7B	1
Gemma 3	1B / 4B	2
Qwen3	1.7B / 4B	2

50 traces per model · identical prompts · 80 / 20 split · 8 ms slots

The label is a model configuration — no weights or prompts are recovered.

What it steals

98.6% — impressive, and bounded



(a) TinyStories 15M/42M/110M (b) Gemma 3 and Qwen 3 (4B)

98.6%

Validation accuracy

8 model choices

Generality beyond identical prompts and fixed runtimes remains open.

Distinguishable configurations within a controlled dataset.

Does it hold up?

It barely flinches under stress

METRIC	LOCAL	NOISY LOCAL	CLOUD	NOISY CLOUD
DevTLB covert (kbps)	17.19	16.81	16.97	16.52
SWQ covert (kbps)	4.02	4.08	3.96	3.80
Website accuracy	85.7%	86.0%	85.5%	85.1%
DevTLB keystroke F1	92.0%	90.5%	93.4%	93.0%
SWQ keystroke F1	98.4%	98.2%	99.1%	98.9%
LLM accuracy	98.6%	98.6%	97.7%	98.0%

Robustness covers these stressors; broader tenant interference remains open.

Does it hold up?

Better on two axes — honestly, not all

WORK / MECHANISM	CAPACITY	BIT ERROR	KEYSTROKE F1	KEYSTROKE Σ	WEBSITE ACC.
IPI	3.45 kbps	18.9%	97.9%	6.15 ms	— (F1 80.4%)
DEVIOUS / IOTLB	2.16 kbps	2.18%	—	10.08 ms	98.9% [1]
This work / DevTLB	**17.19 kbps**	**4.63%**	**92.0%**	**5.29 ms**	**85.7%**
This work / SWQ	**4.02 kbps**	**13.11%**	**98.4%**	**1.21 ms**	**—**

[1] *DEVIOUS traces include GPU fingerprints; its IOTLB channel is since mitigated by PASID. IPI's keystroke F1 beats DevTLB; DEVIOUS's bit error beats both.*

Best on capacity (DevTLB) and keystroke timing (SWQ) — not on every axis.

Does it hold up?

The clean hardware fix costs the thing DSA is for

DevTLB

Partition replacement state by PASID

Cost: capacity · area · utilization

Goal: block cross-tenant eviction

SWQ

Make queue-state feedback privileged

Challenge: legit submission retries

Goal: block occupancy inference

A defense must constrain interference and feedback, not just permissions.

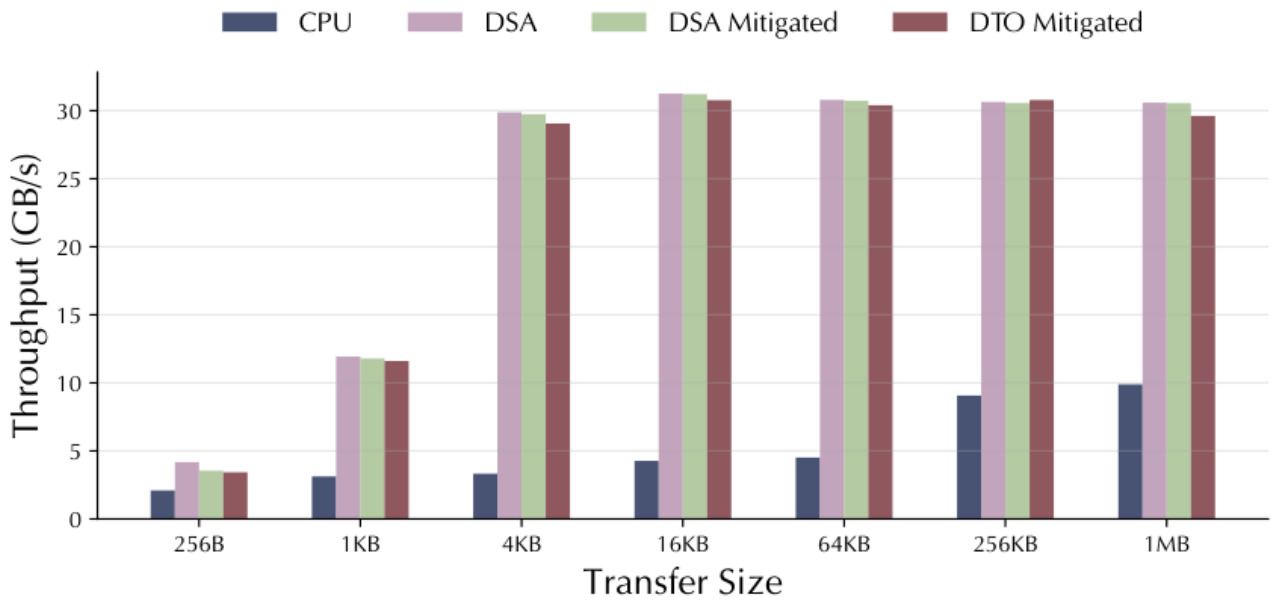
What an operator can do right now

EXPOSURE	BOUNDARY	SOFTWARE RESPONSE
DevTLB eviction	DSA engine	Allocate engines by trust domain
SWQ admission feedback	shared work queue	Avoid cross-tenant SWQ sharing
Sensitive-op offload	application path	Disable DSA on critical paths if needed

Separate queues do not close the shared-engine DevTLB channel.

Does it hold up?

They measured the cost — not the leftover leak



Maximum reported loss

Native DSA	15.7%
DTO	17.9%
At	256-byte transfers

The plot measures throughput overhead; residual leakage is not quantified.

Conclusion

Findings and limits

- **Channels**

  DevTLB replacement timing + SWQ admission feedback

- **Evidence**

  Controlled page / field / engine tests and four workload demonstrations

- **Scope**

  Provisioned co-location; controlled datasets; residual mitigation leakage unmeasured

Shared accelerator state belongs in the isolation model.

Glossary

TERM	MEANING IN THIS PAPER
DSA	Intel Data Streaming Accelerator
ATS / TA	Address Translation Services / Translation Agent
ATC / DevTLB	device-side cache of address translations
PASID	process address-space ID; translation context
WQ / SWQ / DWQ	work queue / shared / dedicated work queue
DMWr / ENQCMD	deferrable memory write / descriptor submission
DTO	DSA Transparent Offload library

Observation windows have different constraints

ASPECT	DEVTLB	SWQ
Prepare	own comp-page cached	effective occupancy ≈ N−1
Idle window	permit victim execution	shorter than holding copy
Probe	time own completion	read acceptance / retry
Calibration	hit / miss distributions	queue depth + drain
Confounder	other users of the slot	other submissions / completions

Keystroke count arithmetic needs clarification

FOR 512 GROUND-TRUTH EVENTS	DEVTLB	SWQ
Reported detections / FP	515 / 15	511 / 7
Implied TP = det − FP	500	504
Implied FN = 512 − TP	12	8
Printed FN / TP	**61 / 500**	**9 / 507**
Diagnostic F1 (fixed totals)	97.37%	98.53%

These calculations expose the inconsistency; they do not repair the experiment. Also: the abstract/contributions say 4.64% covert bit error; §VI-A and Table IV say 4.63%.

Sampling and dataset sizes

SETTING	WEBSITE TASK	LLM TASK
Sample interval	10 µs	10 µs
Samples per slot	400	800
Slot duration	4 ms	8 ms
Classes × traces	100 × 200	8 × 50
Total traces	20,000	400
80 / 20 split	16,000 / 4,000	320 / 80

Reproduction requires a configured DSA platform

- **Hardware & provisioning**

  DSA-capable Xeon; VT-d / IDXD; admin configuration + device access

- **Victim-specific software**

  VPP / DPDK for websites; DTO-enabled SSH and inference workloads

- **Artifact organization**

  playground → characterization; attack → traces; evaluate → metrics

github.com/hkustgz-secarch/dsassassin · DOI 10.5281/zenodo.17799273

Likely questions and concise answers

QUESTION	ANSWER
Does encryption stop it?	content protection does not remove activity patterns
Do separate CPU cores help?	the observations arise in shared DSA resources
Do separate queues help?	SWQ: yes; DevTLB: also separate the engines
Are typed characters recovered?	it evaluates event timing, not full text recovery
Would timer removal solve it?	SWQ reads admission feedback without a duration
Is random insertion a proven fix?	overhead is measured; residual leakage is not

Sources and provenance

- **Primary paper**

  Chen, Li, Deng, Wang, Chen. DSASSASSIN. HPCA 2026

- **Author-hosted PDF**

  i.benx.dev/assets/hpca26dsassassin.pdf

- **Artifact**

  github.com/hkustgz-secarch/dsassassin

- **Archive + record**

  doi.org/10.5281/zenodo.17799273 · 2026.hpca-conf.org